

Student Name :Disha Dipak Naik Batch

:CS02

Roll Number :CS35

Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction on the Breast Cancer Wisconsin (Diagnostic) dataset, visualize the high-dimensional feature space in a reduced 2D plane, and evaluate its impact on classification performance.

Objectives

1. To understand the concepts of dimensionality reduction, feature correlation, and Principal Component Analysis.
2. To preprocess, clean, and scale high-dimensional real-world data using StandardScaler.
3. To implement PCA using Python to compress feature dimensions from 30 continuous features down to 2 principal components.
4. To calculate and plot the explained variance ratio and cumulative explained variance.
5. To visualize the transformed 2D dataset and analyze class separability.
6. To evaluate and compare classification model performance (e.g., Logistic Regression) before and after applying PCA.

Relevant Theory

Support Vector Machine

Principal Component Analysis (PCA)Principal Component Analysis (PCA) is an unsupervised machine learning technique used for dimensionality reduction. It transforms a dataset containing a large set of correlated features into a smaller set of uncorrelated variables called Principal Components (PCs), while preserving as much data variance as possible.

Mathematical Steps:

1. Standardization: Scale each feature to have a mean of 0 ($\mu = 0$) and standard deviation of 1 ($\sigma = 1$).
2. Covariance Matrix: Compute the covariance matrix Σ to quantify feature correlations:

$$\lambda = \frac{1}{\lambda - 1} \lambda^2 \lambda$$

3. Eigen decomposition: Compute eigenvalues (λ) and eigenvectors (v) of matrix Σ :

$$\Sigma v = \lambda v$$

- Eigenvectors represent the direction of the principal axes.
- Eigenvalues indicate the amount of variance along those axes.

4. Projection: Project original data X onto top k principal component directions v :

$$X_{pca} = X \cdot v$$

Dataset Information

1. Dataset Name: Breast Cancer Wisconsin (Diagnostic) Dataset
2. Kaggle Link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
3. Classes: 2 binary classes (Malignant [1] and Benign [0])
4. Features: 30 continuous features computed from digitized images of fine needle aspirates (FNA) of a breast mass (e.g., radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, concavity_mean, etc.).
5. Non-Feature Columns: id (identifier) and Unnamed: 32 (empty column).

Task 1: Import Libraries and Load Dataset

Import core Python libraries (numpy, pandas, matplotlib, seaborn, sklearn) and load data.csv obtained from Kaggle.

```
# Import required Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Import required Scikit-Learn Libraries
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix

# Load the dataset
df = pd.read_csv("data.csv")

# Display first 5 rows
print("First 5 rows of the dataset:")
print(df.head())
```

```
# Display dataset shape
print("\nDataset Shape:")
print(df.shape)
```

```
# Display column names
print("\nColumn Names:")
print(df.columns)
```

First 5 rows of the dataset:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	texture_worst	perimeter_worst	area_worst	smoothness_worst
0	17.33	184.60	2019.0	0.1622
1	23.41	158.80	1956.0	0.1238
2	25.53	152.50	1709.0	0.1444
3	26.50	98.87	567.7	0.2098
4	16.67	152.20	1575.0	0.1374

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst
0	0.6656	0.7119	0.2654	0.4601
1	0.1866	0.2416	0.1860	0.2750
2	0.4245	0.4504	0.2430	0.3613
3	0.8663	0.6869	0.2575	0.6638
4	0.2050	0.4000	0.1625	0.2364

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Dataset Shape:
(569, 33)

Column Names:

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

Task 2: Explore the Dataset

Inspect missing values, view dataset summary statistics, check column data types, and inspect target class distributions (diagnosis).

```
# Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Display summary statistics
print("\nSummary Statistics:")
print(df.describe())

# Check data types of all columns
print("\nColumn Data Types:")
print(df.dtypes)

# Check diagnosis class distribution
print("\nDiagnosis Class Distribution:")
print(df["diagnosis"].value_counts())

# Display percentage of each class
print("\nDiagnosis Class Percentage:")
print(df["diagnosis"].value_counts(normalize=True) * 100)
```

```
Missing Values:
id                0
diagnosis         0
radius_mean      0
texture_mean     0
perimeter_mean   0
area_mean        0
smoothness_mean  0
compactness_mean 0
concavity_mean   0
concave points_mean
symmetry_mean    0
fractal_dimension_mean
radius_se        0
texture_se       0
perimeter_se     0
area_se          0
smoothness_se    0
compactness_se   0
concavity_se     0
concave points_se
symmetry_se      0
fractal_dimension_se
radius_worst     0
texture_worst    0
perimeter_worst  0
area_worst       0
smoothness_worst 0
compactness_worst
concavity_worst  0
concave points_worst
symmetry_worst   0
fractal_dimension_worst
Unnamed: 32      569
dtype: int64
```

Summary Statistics:

	id	radius_mean	texture_mean	perimeter_mean	area_mean
\					
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000

	smoothness_mean	compactness_mean	concavity_mean	concave	points_mean
\					
count	569.000000	569.000000	569.000000		569.000000
mean	0.096360	0.104341	0.088799		0.048919
std	0.014064	0.052813	0.079720		0.038803
min	0.052630	0.019380	0.000000		0.000000
25%	0.086370	0.064920	0.029560		0.020310
50%	0.095870	0.092630	0.061540		0.033500
75%	0.105300	0.130400	0.130700		0.074000
max	0.163400	0.345400	0.426800		0.201200

	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	\
count	569.000000	...	569.000000	569.000000	569.000000	
mean	0.181162	...	25.677223	107.261213	880.583128	
std	0.027414	...	6.146258	33.602542	569.356993	
min	0.106000	...	12.020000	50.410000	185.200000	
25%	0.161900	...	21.080000	84.110000	515.300000	
50%	0.179200	...	25.410000	97.660000	686.500000	
75%	0.195700	...	29.720000	125.400000	1084.000000	
max	0.304000	...	49.540000	251.200000	4254.000000	

	smoothness_worst	compactness_worst	concavity_worst	\
count	569.000000	569.000000	569.000000	
mean	0.132369	0.254265	0.272188	
std	0.022832	0.157336	0.208624	
min	0.071170	0.027290	0.000000	
25%	0.116600	0.147200	0.114500	
50%	0.131300	0.211900	0.226700	
75%	0.146000	0.339100	0.382900	
max	0.222600	1.058000	1.252000	

	concave	points_worst	symmetry_worst	fractal_dimension_worst	\
count		569.000000	569.000000	569.000000	
mean		0.114606	0.290076	0.083946	
std		0.065732	0.061867	0.018061	
min		0.000000	0.156500	0.055040	
25%		0.064930	0.250400	0.071460	
50%		0.099930	0.282200	0.080040	
75%		0.161400	0.317900	0.092080	
max		0.291000	0.663800	0.207500	

	Unnamed: 32
count	0.0
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

[8 rows x 32 columns]

```

Column Data Types:
id                int64
diagnosis         object
radius_mean      float64
texture_mean     float64
perimeter_mean   float64
area_mean        float64
smoothness_mean  float64
compactness_mean float64
concavity_mean   float64
concave points_mean float64
symmetry_mean    float64
fractal_dimension_mean float64
radius_se        float64
texture_se       float64
perimeter_se     float64
area_se          float64
smoothness_se    float64
compactness_se   float64
concavity_se     float64
concave points_se float64
symmetry_se      float64
fractal_dimension_se float64
radius_worst     float64
texture_worst    float64
perimeter_worst  float64
area_worst       float64
smoothness_worst float64
compactness_worst float64
concavity_worst  float64
concave points_worst float64
symmetry_worst   float64
fractal_dimension_worst float64
Unnamed: 32      float64
dtype: object

```

Diagnosis Class Distribution:

```

diagnosis
B      357
M      212
Name: count, dtype: int64

```

Diagnosis Class Percentage:

```

diagnosis
B      62.741652
M      37.258348
Name: proportion, dtype: float64

```

Task 3: Data Cleaning and Feature/Target Selection

Drop metadata columns (id and Unnamed: 32). Convert the categorical target column (diagnosis: 'M'/'B') into numerical values (1/0) and separate feature matrix X (30 features) from label vector y .

```
# Task 3: Data Cleaning and Feature/Target Selection
```

```
# Drop unnecessary columns
```

```
df = df.drop(columns=["id", "Unnamed: 32"])
```

```

# Convert diagnosis into numerical values
# M = 1 (Malignant), B = 0 (Benign)
df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})

# Separate features and target
X = df.drop(columns=["diagnosis"])
y = df["diagnosis"]

# Display shapes
print("Feature Matrix X shape:", X.shape)
print("Target Vector y shape:", y.shape)

# Display first 5 rows of X
print("\nFeatures:")
print(X.head())

# Display first 5 values of y
print("\nTarget:")
print(y.head())

```

Feature Matrix X shape: (569, 30)
Target Vector y shape: (569,)

Features:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	compactness_mean	concavity_mean	concave	points_mean	symmetry_mean	\
0	0.27760	0.3001		0.14710	0.2419	
1	0.07864	0.0869		0.07017	0.1812	
2	0.15990	0.1974		0.12790	0.2069	
3	0.28390	0.2414		0.10520	0.2597	
4	0.13280	0.1980		0.10430	0.1809	

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst	\
0		0.07871 ...	25.38	17.33	184.60	
1		0.05667 ...	24.99	23.41	158.80	
2		0.05999 ...	23.57	25.53	152.50	
3		0.09744 ...	14.91	26.50	98.87	
4		0.05883 ...	22.54	16.67	152.20	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	concave	points_worst	symmetry_worst	fractal_dimension_worst
0		0.2654	0.4601	0.11890
1		0.1860	0.2750	0.08902
2		0.2430	0.3613	0.08758
3		0.2575	0.6638	0.17300
4		0.1625	0.2364	0.07678

[5 rows x 30 columns]

```
Target:
0      1
1      1
2      1
3      1
4      1
Name: diagnosis, dtype: int64
```

Task 4: Standardize Features

Apply StandardScaler to transform all 30 numeric features so they have zero mean and unit variance ($\mu = 0$, $\sigma = 1$).

```
# Task 4: Standardize Features
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Create StandardScaler object
scaler = StandardScaler()
```

```
# Fit and transform the feature data
X_scaled = scaler.fit_transform(X)
```

```
# Display the shape of scaled data
print("Shape of scaled data:", X_scaled.shape)
```

```
# Display first 5 rows
print("\nFirst 5 rows of scaled data:")
print(X_scaled[:5])
```

Shape of scaled data: (569, 30)

First 5 rows of scaled data:

```
[[ 1.09706398e+00 -2.07333501e+00  1.26993369e+00  9.84374905e-01
  1.56846633e+00  3.28351467e+00  2.65287398e+00  2.53247522e+00
  2.21751501e+00  2.25574689e+00  2.48973393e+00 -5.65265059e-01
  2.83303087e+00  2.48757756e+00 -2.14001647e-01  1.31686157e+00
  7.24026158e-01  6.60819941e-01  1.14875667e+00  9.07083081e-01
  1.88668963e+00 -1.35929347e+00  2.30360062e+00  2.00123749e+00
  1.30768627e+00  2.61666502e+00  2.10952635e+00  2.29607613e+00
  2.75062224e+00  1.93701461e+00]
 [ 1.82982061e+00 -3.53632408e-01  1.68595471e+00  1.90870825e+00
 -8.26962447e-01 -4.87071673e-01 -2.38458552e-02  5.48144156e-01
  1.39236330e-03 -8.68652457e-01  4.99254601e-01 -8.76243603e-01
  2.63326966e-01  7.42401948e-01 -6.05350847e-01 -6.92926270e-01
 -4.40780058e-01  2.60162067e-01 -8.05450380e-01 -9.94437403e-02
  1.80592744e+00 -3.69203222e-01  1.53512599e+00  1.89048899e+00
 -3.75611957e-01 -4.30444219e-01 -1.46748968e-01  1.08708430e+00
 -2.43889668e-01  2.81189987e-01]
 [ 1.57988811e+00  4.56186952e-01  1.56650313e+00  1.55888363e+00
  9.42210440e-01  1.05292554e+00  1.36347845e+00  2.03723076e+00
  9.39684817e-01 -3.98007910e-01  1.22867595e+00 -7.80083377e-01
  8.50928301e-01  1.18133606e+00 -2.97005012e-01  8.14973504e-01
  2.13076435e-01  1.42482747e+00  2.37035535e-01  2.93559404e-01
  1.51187025e+00 -2.39743838e-02  1.34747521e+00  1.45628455e+00
  5.27407405e-01  1.08293217e+00  8.54973944e-01  1.95500035e+00
  1.15225500e+00  2.01391209e-01]
 [-7.68909287e-01  2.53732112e-01 -5.92687167e-01 -7.64463792e-01
  3.28355348e+00  3.40290899e+00  1.91589718e+00  1.45170736e+00]
```

```

2.86738293e+00  4.91091929e+00  3.26373441e-01 -1.10409044e-01
2.86593405e-01 -2.88378148e-01  6.89701660e-01  2.74428041e+00
8.19518384e-01  1.11500701e+00  4.73268037e+00  2.04751088e+00
-2.81464464e-01  1.33984094e-01 -2.49939304e-01 -5.50021228e-01
3.39427470e+00  3.89339743e+00  1.98958826e+00  2.17578601e+00
6.04604135e+00  4.93501034e+00]
[ 1.75029663e+00 -1.15181643e+00  1.77657315e+00  1.82622928e+00
 2.80371830e-01  5.39340452e-01  1.37101143e+00  1.42849277e+00
-9.56046689e-03 -5.62449981e-01  1.27054278e+00 -7.90243702e-01
 1.27318941e+00  1.19035676e+00  1.48306716e+00 -4.85198799e-02
 8.28470780e-01  1.14420474e+00 -3.61092272e-01  4.99328134e-01
 1.29857524e+00 -1.46677038e+00  1.33853946e+00  1.22072425e+00
 2.20556166e-01 -3.13394511e-01  6.13178758e-01  7.29259257e-01
-8.68352984e-01 -3.97099619e-01]]

```

Task 5: Implement PCA (30D to 2D)

Use `sklearn.decomposition.PCA` to reduce feature dimensions from 30 down to 2 principal components ($\mathbb{R}^2_1$ and $\mathbb{R}^2_2$).

Task 5: Implement PCA (30D to 2D)

```
from sklearn.decomposition import PCA
```

Create PCA object with 2 components

```
pca = PCA(n_components=2)
```

Transform the standardized data

```
X_pca = pca.fit_transform(X_scaled)
```

Display the shape

```
print("Original data shape:", X_scaled.shape)
```

```
print("PCA data shape:", X_pca.shape)
```

Display first 5 rows

```
print("\nFirst 5 rows of PCA transformed data:")
```

```
print(X_pca[:5])
```

Original data shape: (569, 30)

PCA data shape: (569, 2)

First 5 rows of PCA transformed data:

```

[[ 9.19283683  1.94858307]
 [ 2.3878018  -3.76817174]
 [ 5.73389628 -1.0751738 ]
 [ 7.1229532  10.27558912]
 [ 3.93530207 -1.94807157]]

```

Task 6: Analyze Explained Variance Ratio

Compute and print the variance explained by $\mathbb{R}^2_1$ and $\mathbb{R}^2_2$, along with the cumulative variance retained.

Task 6: Analyze Explained Variance Ratio

Get explained variance ratio

```
explained_variance = pca.explained_variance_ratio_
```

Print variance explained by each principal component

```
print("Variance explained by PC1:", explained_variance[0])
```

```
print("Variance explained by PC2:", explained_variance[1])
```

Calculate cumulative variance

```
cumulative_variance = explained_variance.cumsum()
```

```
print("\nCummulative variance retained:")
```

```
print(cumulative_variance)
```

Print percentage values

```
print("\nPC1 Variance: {:.2f}%".format(explained_variance[0] * 100))
```

```
print("PC2 Variance: {:.2f}%".format(explained_variance[1] * 100))
```

```
print("Cumulative Variance: {:.2f}%".format(cumulative_variance[-1] * 100))
```

Variance explained by PC1: 0.44272025607526366

Variance explained by PC2: 0.18971182044033078

Cummulative variance retained:

[0.44272026 0.63243208]

PC1 Variance: 44.27%

PC2 Variance: 18.97%

Cummulative Variance: 63.24%

Task 7: Visualize PCA Transformed Boundary / Scatter Plot

Create a 2D scatter plot with $\mathbb{R}^2_1$ on the x-axis and $\mathbb{R}^2_2$ on the y-axis, hue-coded by tumor diagnosis (Malignant vs. Benign).

Task 7: Visualize PCA Transformed Data

```
plt.figure(figsize=(8, 6))
```

Scatter plot

```
sns.scatterplot(
    x=X_pca[:, 0],
    y=X_pca[:, 1],
    hue=y,
    palette=["blue", "red"],
    s=60
)
```

Add Labels and title

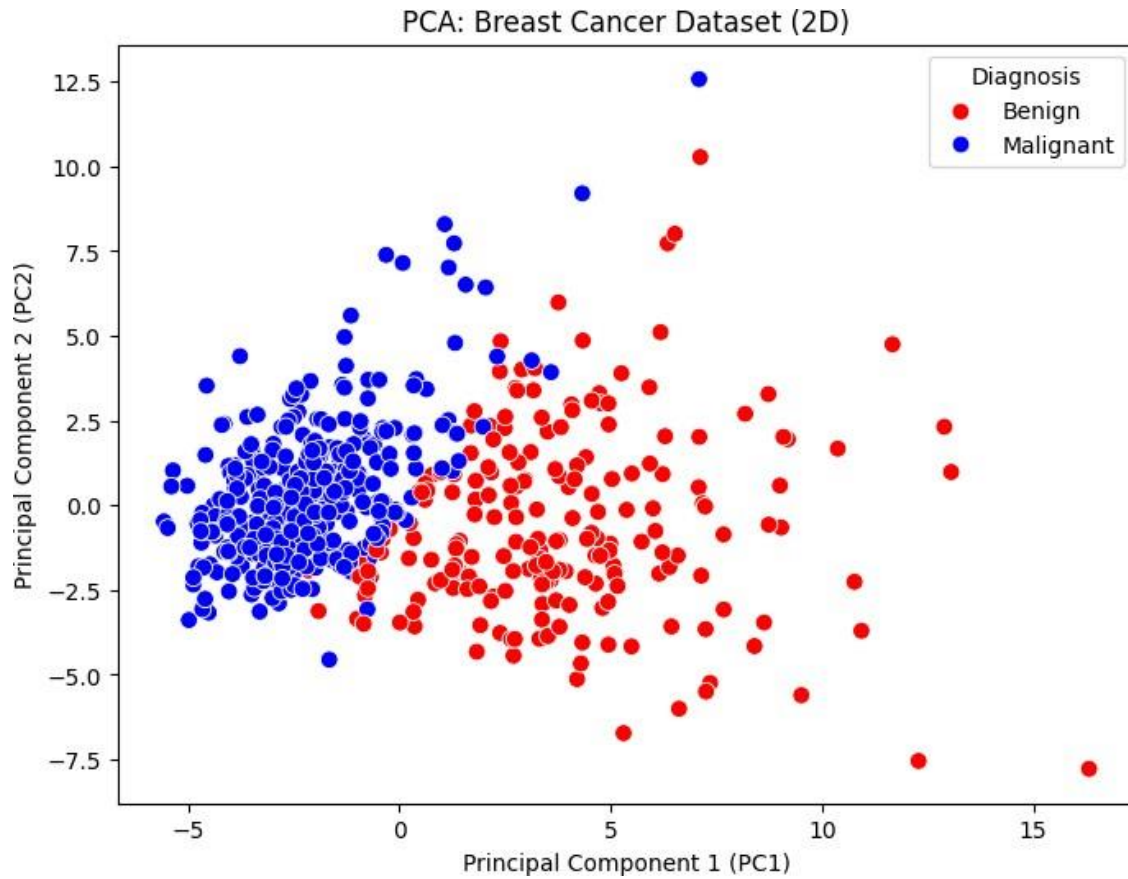
```
plt.xlabel("Principal Component 1 (PC1)")
```

```
plt.ylabel("Principal Component 2 (PC2)")
```

```
plt.title("PCA: Breast Cancer Dataset (2D)")
```

```
plt.legend(title="Diagnosis", labels=["Benign", "Malignant"])
```

```
plt.show()
```



Task 8: Train Classifier on Raw High-Dimensional Data (30 Features)

Split the original scaled 30-feature dataset into training and testing sets. Train a Logistic Regression model and obtain baseline performance metrics.

```
# Task 8: Train Logistic Regression on Raw 30 Features
```

```
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42, stratify=y  
)
```

```
# Create Logistic Regression model
```

```
logistic_model = LogisticRegression(max_iter=1000)
```

```
# Train the model
```

```
logistic_model.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred = logistic_model.predict(X_test)
```

```
# Calculate accuracy
```

```

accuracy = accuracy_score(y_test, y_pred)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("\nBaseline Accuracy:", accuracy)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

Training data shape: (455, 30)
Testing data shape: (114, 30)

Baseline Accuracy: 0.9649122807017544

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.99	0.97	72
1	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Confusion Matrix:

```

[[71  1]
 [ 3 39]]

```

Task 9: Train Classifier on PCA Transformed Data (2 Features)

Split the 2-component PCA dataset into training and testing sets. Train a Logistic Regression model using only $\mathbb{P}_1$ and $\mathbb{P}_2$.

Task 9: Visual Confusion Matrix

```

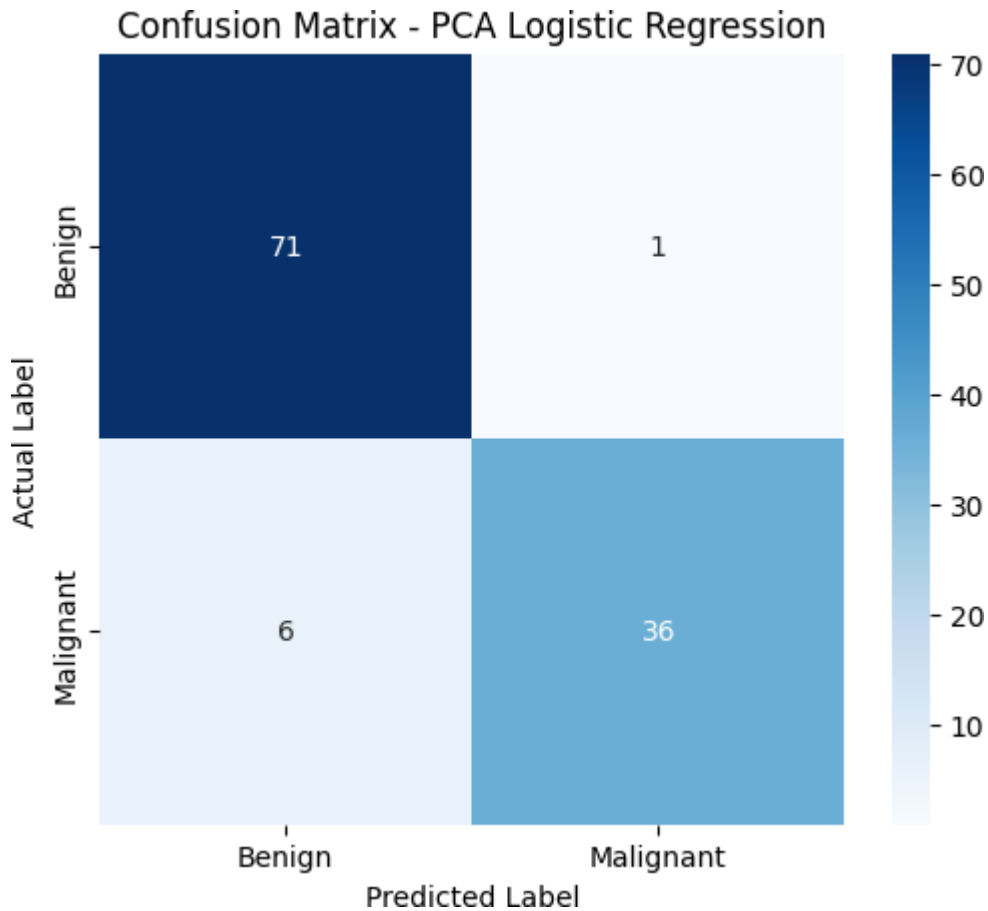
cm_pca = confusion_matrix(y_test_pca, y_pca_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_pca,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")

```

```
plt.title("Confusion Matrix - PCA Logistic Regression")
plt.show()
```



Task 10: Compare Classifier Performance

Compare the performance of the model before PCA (30 features) vs. after PCA (2 components) using:

- Accuracy Score
- Confusion Matrix
- Classification Report (Precision, Recall, F1-Score)

Task 10: Visual Confusion Matrices

Confusion matrix before PCA

```
cm_before = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(
    cm_before,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)
```

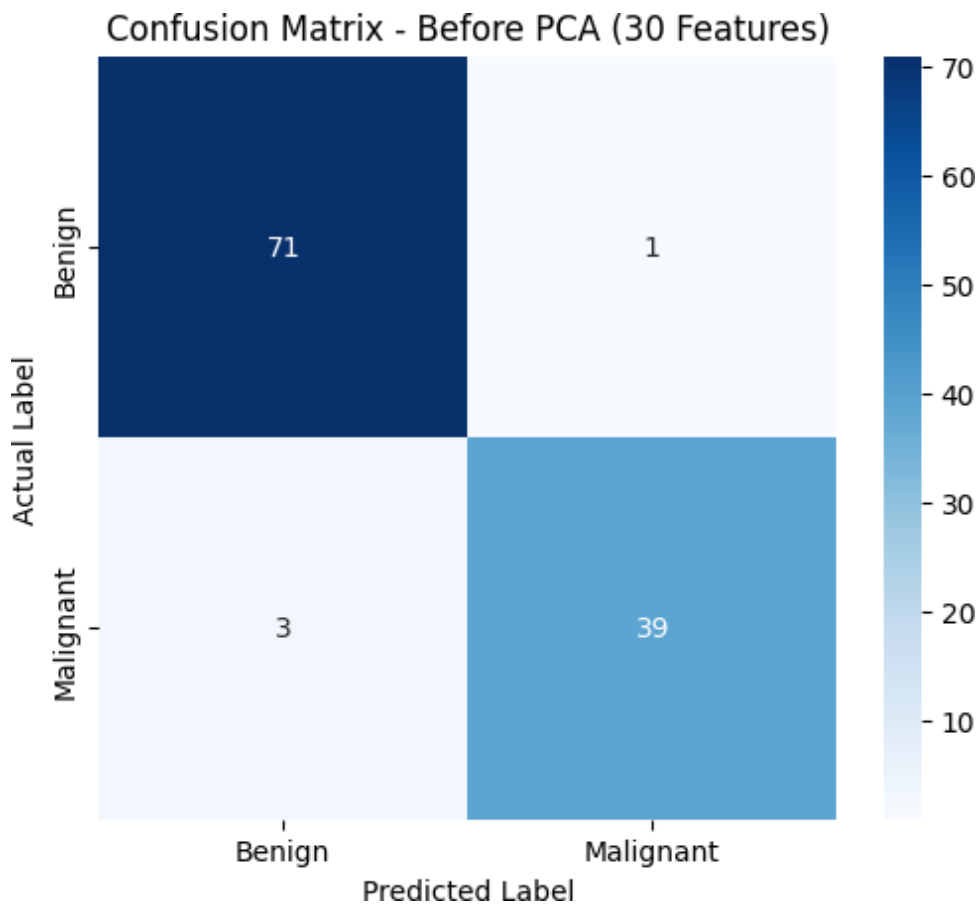
```
)

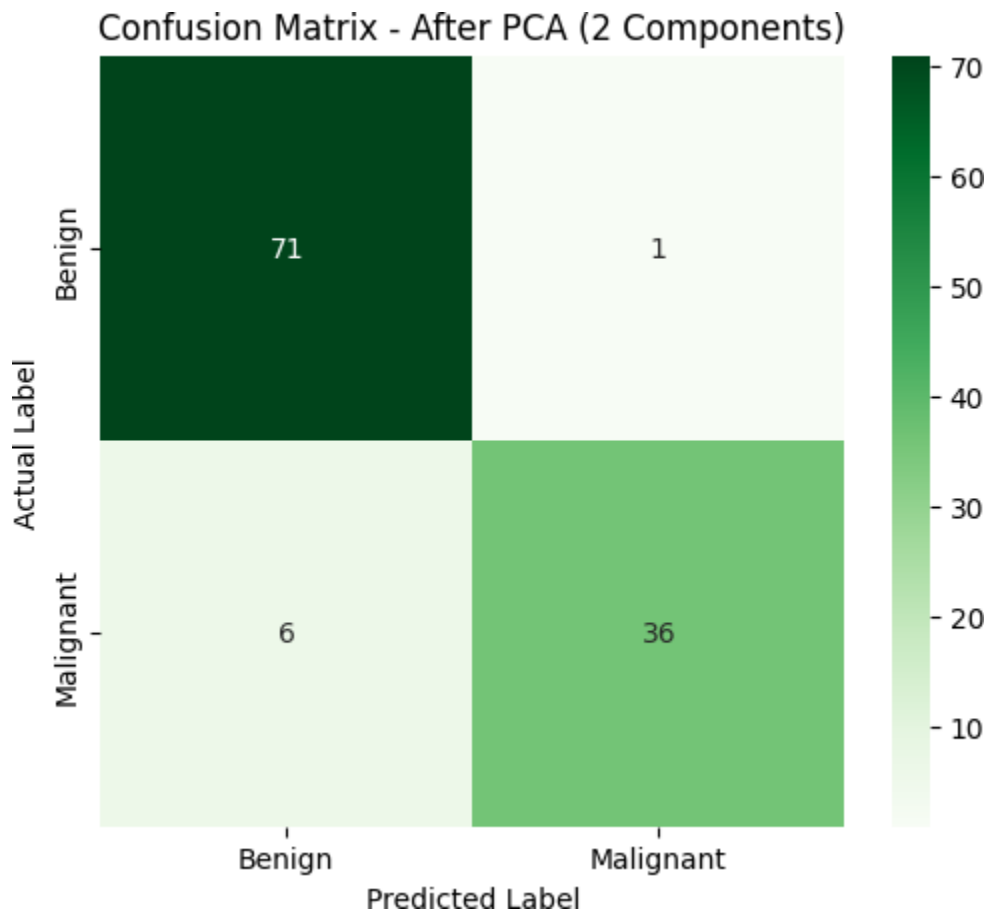
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Before PCA (30 Features)")
plt.show()
```

```
# Confusion matrix after PCA
cm_after = confusion_matrix(y_test_pca, y_pca_pred)
```

```
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm_after,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Benign", "Malignant"],
    yticklabels=["Benign", "Malignant"]
)
```

```
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - After PCA (2 Components)")
plt.show()
```





Conclusion

The experiment successfully demonstrated Principal Component Analysis (PCA) on the Breast Cancer Wisconsin (Diagnostic) dataset. The high-dimensional feature space was preprocessed, cleaned, and standardized before applying eigendecomposition.

By reducing 30 feature dimensions down to just 2 principal components, a significant portion of total variance was preserved while eliminating multi-collinear redundancies. Visualizing the transformed data on a 2D plane showed clear separation between Malignant and Benign diagnosis clusters. Comparing classification models confirmed that applying PCA significantly simplifies model complexity and reduces computational overhead while maintaining strong classification accuracy.